

[Team Project Report]

Reconfigurable FIR Filter

[조원]

20212995 한승조

20201615 최승완

20212990 정의찬

20232095 유지원

1. 개요

1-1) 전체 개요

Reconfigurable FIR(Finite Impulse Response) Filter는 필터의 계수(coefficient)를 유동적으로 변경하여 작동하는 FIR 구조이다. 기존의 고정 계수 FIR 필터는 설계 시 정해진 필터 특성만을 제공하는 반면, Reconfigurable FIR Filter는 동작 중 또는 재설정 시 계수를 변경함으로써 하나의 하드웨어로 다양한 필터 특성을 구현할 수 있다는 장점을 가진다.

FIR 필터는 본질적으로 안정성(stability)이 보장되고 위상 특성이 선형(linear phase)이라는 특성 때문에 디지털 신호 처리(DSP) 시스템에서 널리 사용된다. 특히, Reconfigurable 구조를 적용함으로써 하드웨어적인 자원 사용을 최소화하면서도 다양한 신호 처리 요구를 만족시킬 수 있다.

본 프로젝트에서는 FIR 필터의 계수를 외부 메모리(SRAM)에 저장하고, 제어 로직을 통해 선택적으로 로드함으로써 필터 특성을 재구성할 수 있는 구조를 설계하였다.

1-2) 동작 원리

Reconfigurable FIR Filter는 입력 신호와 계수의 곱을 누적(accumulation)하는 기본적인 FIR 연산 구조를 기반으로 한다.

필터 출력은 다음과 같은 식으로 표현된다.

$$y[n] = \sum_{k=0}^{N-1} h[k] \cdot x[n-k]$$

이때 $h[k]$ 는 필터 계수이며, Reconfigurable FIR Filter에서는 이 계수가 고정되지 않고 필요에 따라 변경 가능하다. 해당 프로젝트에서는 계수 메모리를 SRAM을 통하여 구현하고, 저역통과(low-pass), 고역통과(high-pass), 대역통과(band-pass)와 같은 다양한 필터 특성을 동일한 하드웨어 구조에서 선택적으로 구현할 수 있도록 하였다.

1-3) 목적

본 프로젝트의 목적은 Reconfigurable FIR Filter를 FPGA 기반으로 설계 및 검증하는 데 있다. 이를 위해 다음과 같은 목표를 설정하였다.

FIR 필터의 기본 구조 및 동작 원리 이해

- SRAM 기반 계수 로딩 구조 설계
- Testbench를 통한 waveform 분석

- 시뮬레이션 결과를 통해 설계의 타당성 검증

이를 통해 본 프로젝트는 이론적 DSP 개념과 실제 하드웨어 설계 간의 연관성을 이해하고, 재구성 가능한 디지털 신호 처리 시스템의 구현 가능성을 확인하는 데 의의를 둔다.

2. 설계 구조

2-1) FIR Filter 사양 및 Enable Signal 정의

2-1-1) FIR Filter 설계 사양 (Specification)

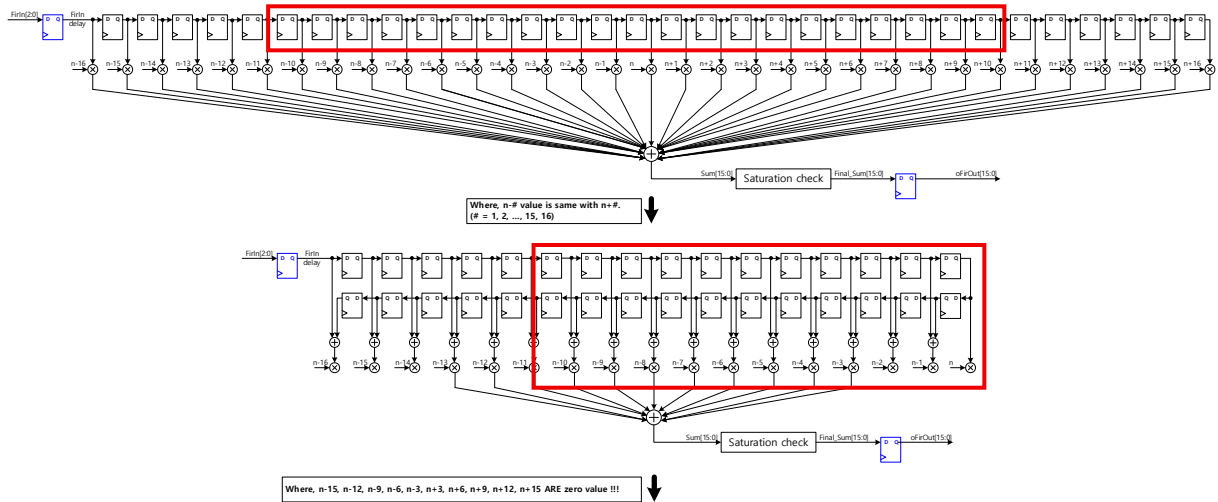
본 프로젝트에서 구현한 Reconfigurable FIR Filter의 주요 사양은 다음과 같다.

항목	사양
Filter Type	FIR (Finite Impulse Response)
Reconfigurability	Coefficient SRAM 기반 재구성 가능
Number of Taps	21 taps
Filter Structure	Multiply-Accumulate (MAC)
Arithmetic	Signed Fixed-Point
Coefficient Storage	SRAM (Addressable)
Clock Domain	Single clock
Output Type	Accumulated FIR output

2-1-2) 입력 / 출력 사양

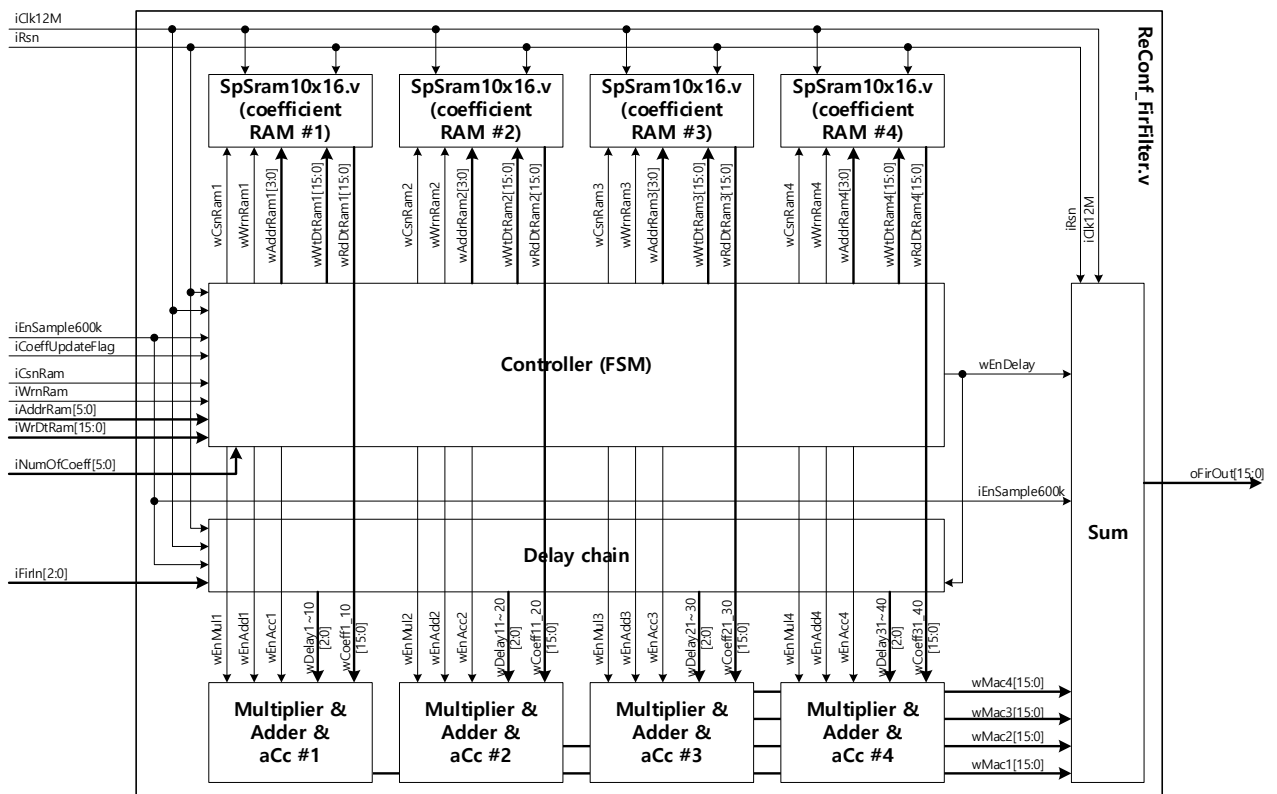
해당 프로젝트에서 사용하는 enable signal들로는 clk(시스템 동작 clock)이고 sampling rate로는 600kHz($1/20 \times \text{clock rate}$)를 사용한다. iFirIn[2:0] pattern은 600kHz/64ea로 64번에 한번 3'b001 또는 3'b111을 입력하게 된다. 나머지 63번은 3'b000을 입력한다. oFirOut[15:0]은 3'b001 impulse response와 3'b111 impulse response의 예상값, 즉 FIR filter의 coefficient value를 출력하게 된다.

2-2) HW block diagram



이때 과제 요구사항으로 21tap만 사용하므로, 표시된 부분만 사용하게 된다.

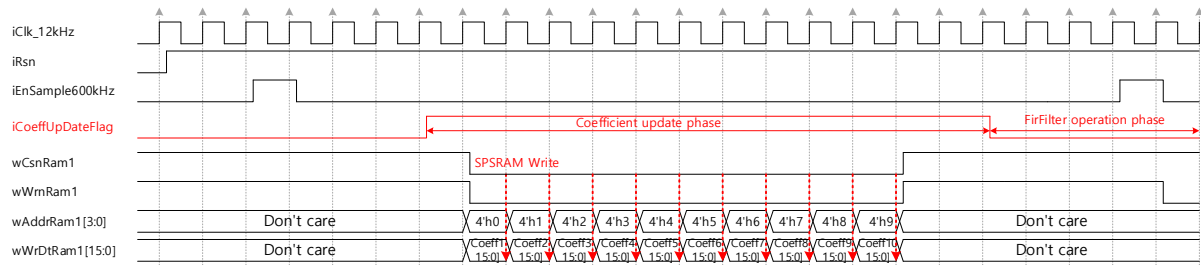
2-3) Block Diagram



기본적인 Block Diagram은 교수님이 제공해주신 설계 구조를 따랐다. Verilog code와 Testbench, Waveform 또한 이에 맞춰 구성, 출력되었다.

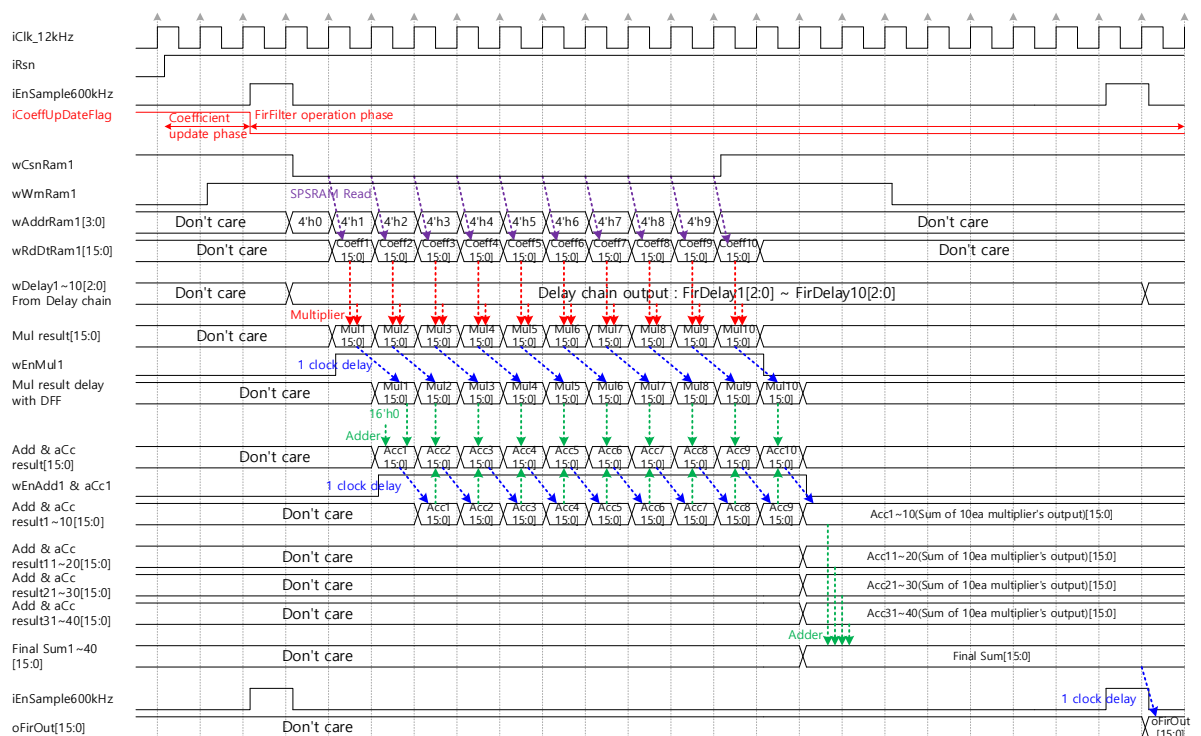
2-4) Timing Diagram

2-4-1) Coefficient update phase



Coefficient update phase는 FIR 연산과는 독립적인 것으로, 필터 계수를 SRAM에 로드하거나 변경하는 구간이다. 즉, FIR 연산이나 tap 누적 단계가 아닌 다음 FIR 동작을 위한 준비 단계라고 볼 수 있다. 이는 Reconfigurable FIR Filter에서는 위에 설명했듯이 계수를 계속해서 바꿀 수 있어야 하는데 FIR 연산 중에 계수를 바꾸면 TAP마다 다른 필터 계수를 사용해야하고, waveform이 고르게 출력되지 않게 된다. 이에 계수 업데이트는 반드시 분리된 타이밍에 진행해야 한다.

2-4-2) Filter operation phase



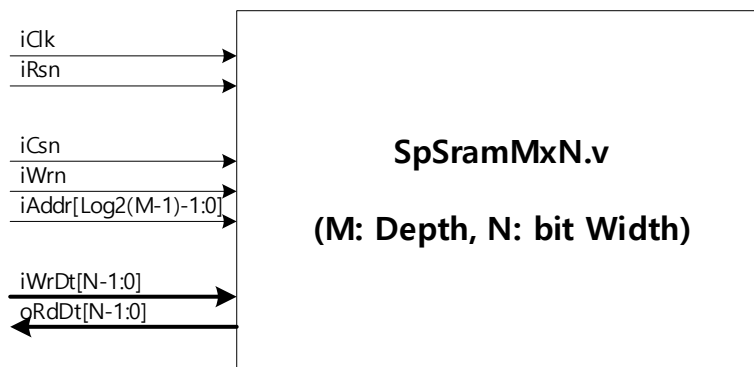
Filter operation phase는 저장된 계수를 이용하여 입력 신호에 대한 FIR 연산(MAC 연산)을 수행하는 구간이다. 이는 계수 쓰거나 필터 구조 변경이 없고 계수를 고정하고 데이터 연산만 수행하는 과정이다. 따라서 Filter operation phase에서는 계수 업데이트가 금지된 상태에서 SRAM에 저장된 계수를 순차적으로 읽어와 입력 데이터와의 곱셈 및 누적 연산을 수행하게 되며, 모든 tap 연산이 완료된 후 출력이 유효화된다.

2-5) SRAM

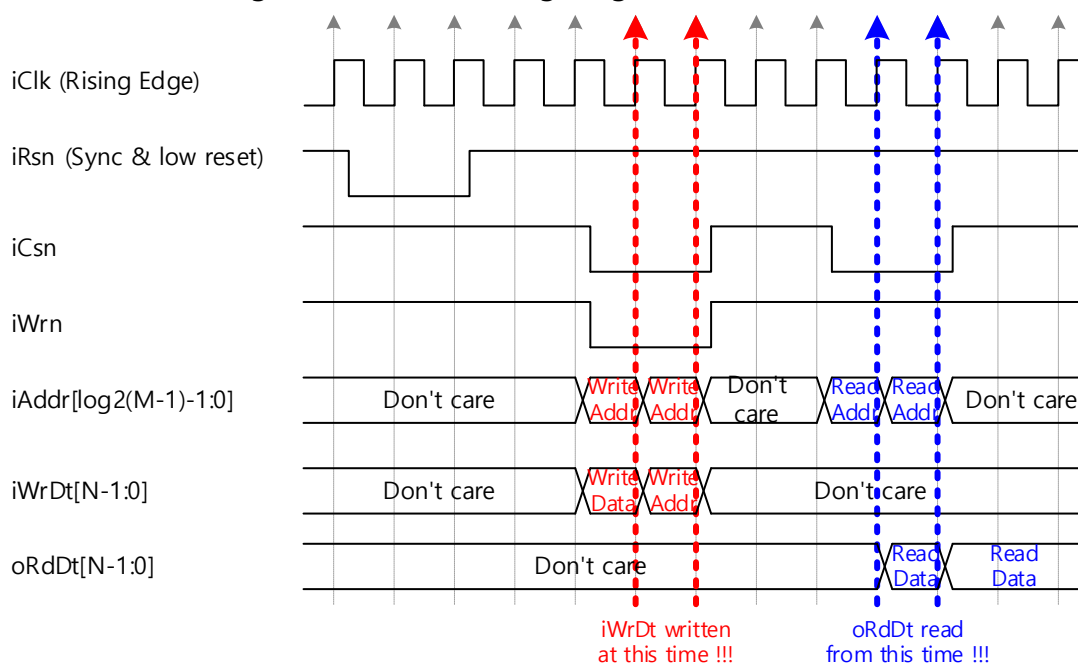
2-5-1) SRAM의 개념

SRAM(Static Random Access Memory)는 flip-flop 기반으로 데이터를 저장하며, refresh가 필요하지 않은 고속 메모리이다. SRAM은 clock에 동기화된 제어신호를 통해 address 기반으로 데이터를 읽고 쓰는 구조를 가진다. DRAM과는 달리 주기적인 데이터 갱신이 필요하지 않으므로 안정적인 읽기, 쓰기 동작이 가능해진다. 즉, 해당 프로젝트에서 SRAM은 FIR 계수인 $h[k]$ 를 저장, Filter Operation Phase 동안 Read-only, Coefficient Update phase 동안 Write-only, FIR Core와 독립된 구조 유지에 사용된다. 따라서, SRAM은 FIR 필터 계수를 저장하기 위한 메모리로 사용되며, 계수 업데이트 단계에서는 쓰기 동작을 수행하고 필터 연산 단계에서는 읽기 동작만 수행함으로써 연산 중 계수 변경을 방지하는 역할을 한다.

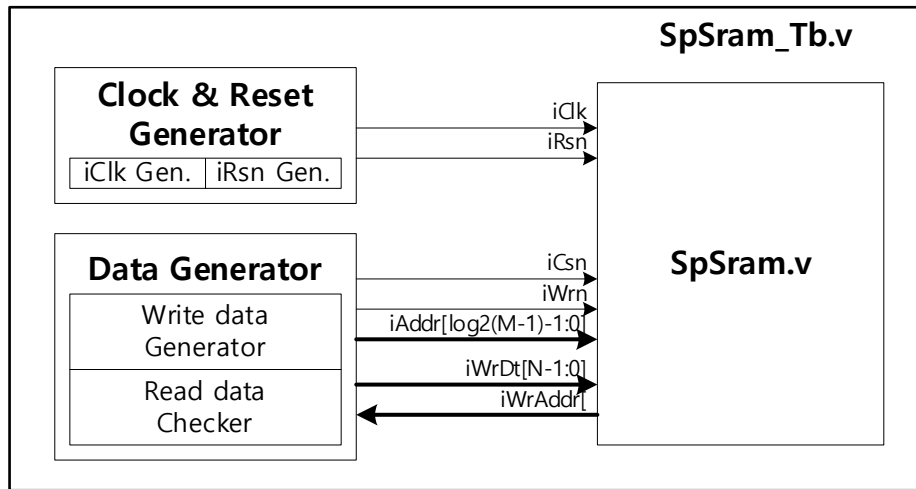
2-5-2) SRAM의 Block Diagram



2-5-3) SPSRAM(Single Port SRAM) Timing Diagram



2-5-4) RTL Modeling & Testbench

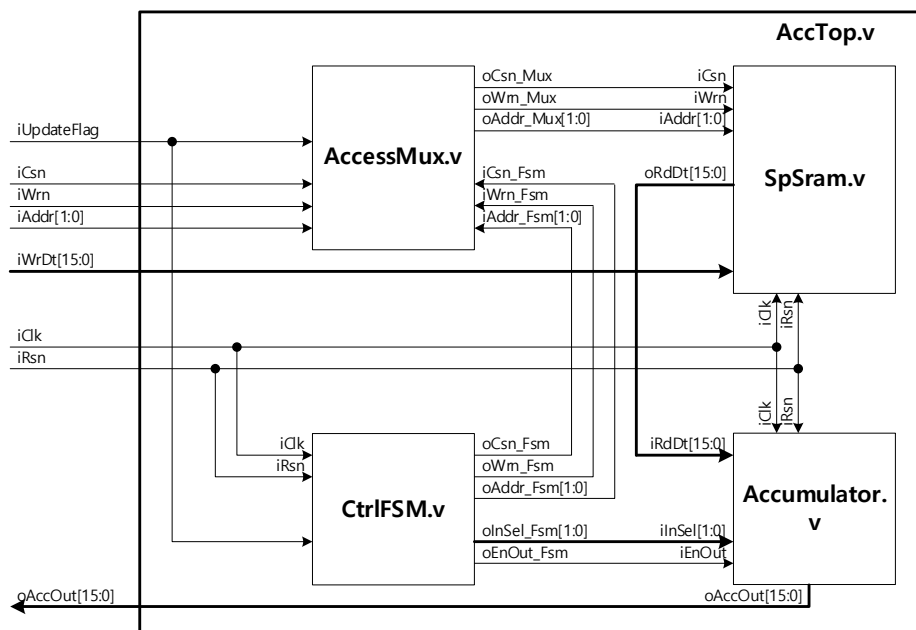


2-6) Accumulator

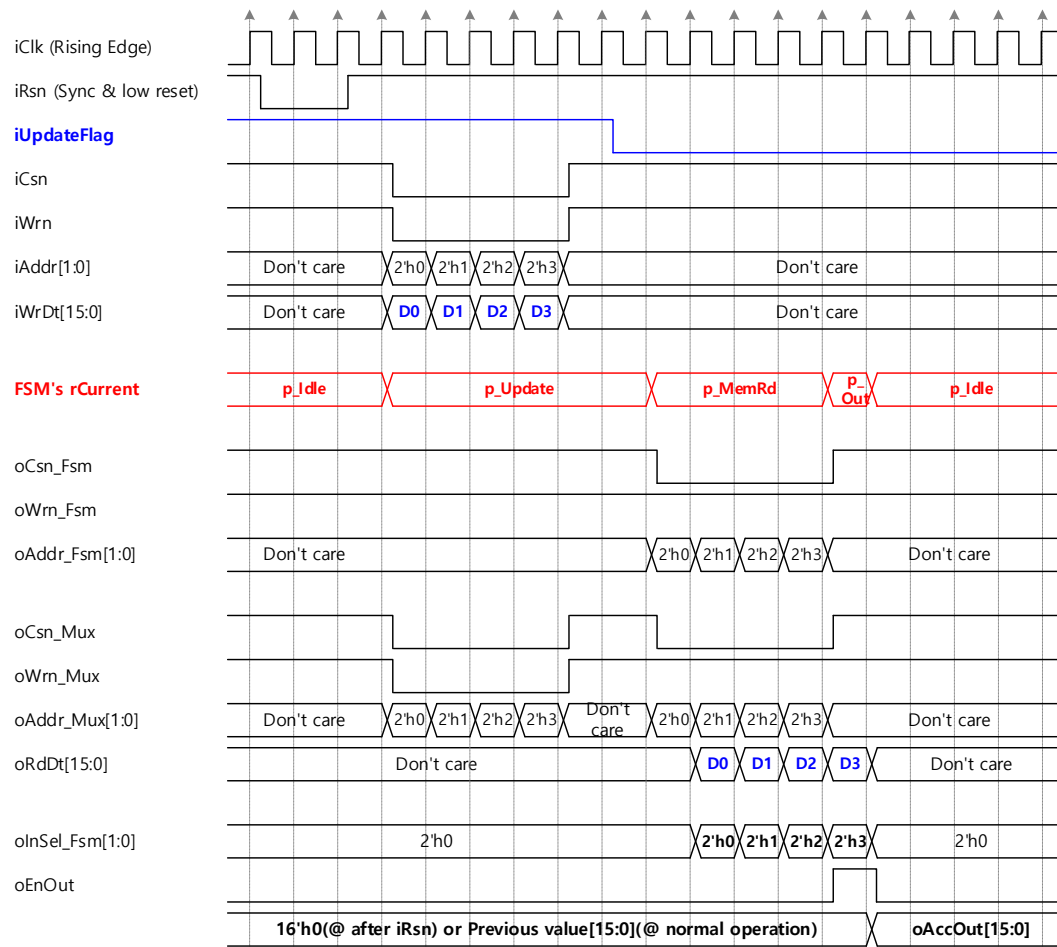
2-6-1) Accumulator의 개념

Accumulator는 FIR 필터에서 각 tap의 곱셈 결과를 누적하여 최종 출력값을 생성하는 순차 논리 블록이다. FIR Filter의 출력은 단일 연산 결과가 아니라 여러 개의 곱셈 결과를 순차적으로 더한 값으로 누적 연산을 수행하는 Accumulator는 필수적인 구성 요소이다. Accumulator는 누적 레지스터(Accumulation Register), 가산기(Adder), 제어신호(enable, clear)의 요소로 이루어진다. 따라서 Accumulator는 각 tap의 곱셈 결과를 순차적으로 누적하여 FIR 필터의 출력 값을 생성하며, 연산 시작 시 초기화 신호를 통해 이전 연산 결과의 영향을 제거한다.

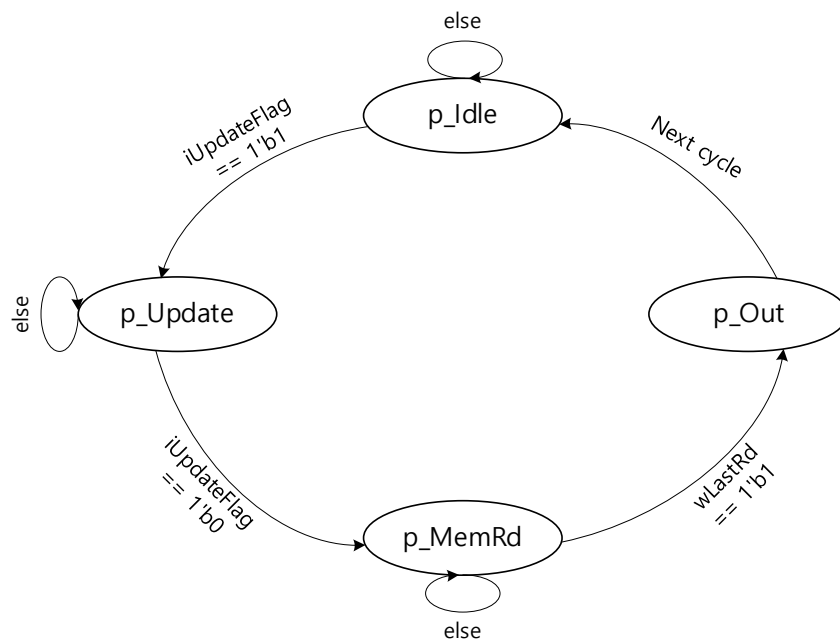
2-6-2) Accumulator Block Diagram



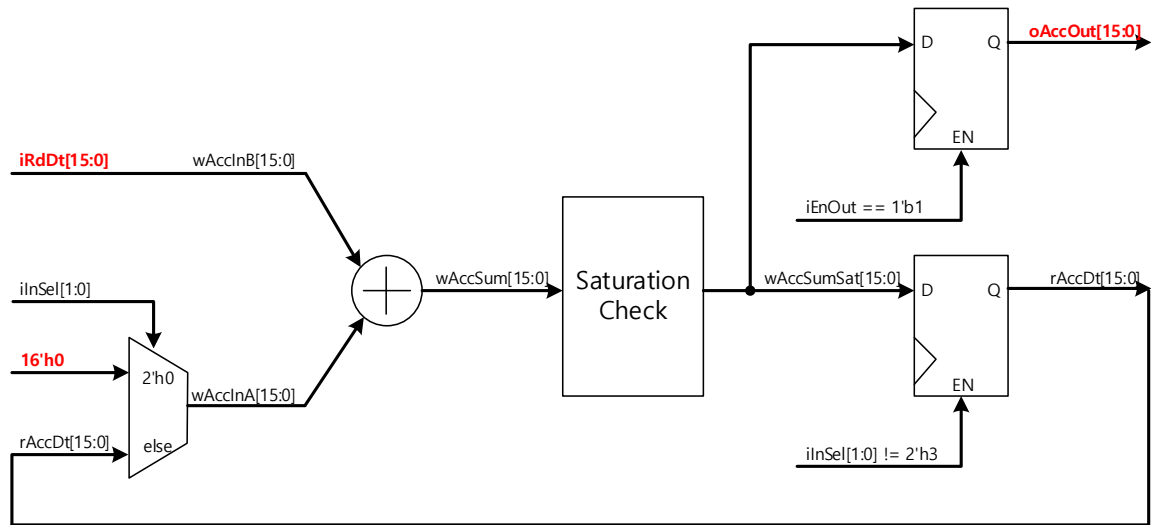
2-6-3) Accumulator Timing Diagram



2-6-4) Accumulator FSM(Finite State Machine)



2-6-5) Accumulator's conceptual Diagram



3. Verilog

3-1) Reconfigurable_FIR_Top

```

module Reconfigurable_FIR_Top (
    input wire iClk12M,
    input wire iRstn,
    output wire signed [15:0] oFirFinalSum
);
    // Enable signals
    wire wEn600k, wEn400k;
    wire [7:0] wSampleCnt, wScramCnt;

    // Data signals
    wire signed [15:0] wImpulseOut;
    wire signed [15:0] wFirOut;
    wire signed [15:0] wAccSum;
    wire [15:0] wDacOut;

    // Enable generator
    EnableGen uEnableGen (
        .iClk12M(iClk12M),
        .iRstn(iRstn),
        .oEn600k(wEn600k),
        .oEn400k(wEn400k),
        .oSampleCnt(wSampleCnt),
        .oScramCnt(wScramCnt)
    );

    // Impulse generator
    ImpulseGen uImpulseGen (
        .iClk(iClk12M),
        .iRstn(iRstn),
        .iEn(wEn600k),
        .oImpulse(wImpulseOut)
    );

    // FIR Filter
    FIR_Filter uFirFilter (
        .iClk(iClk12M),
        .iRstn(iRstn),
        .iEn(wEn600k),
        .iData(wImpulseOut),
        .oData(wFirOut)
    );

    // Accumulator
    Accumulator uAccumulator (
        .iClk(iClk12M),
        .iRstn(iRstn),
        .iEn(wEn400k),
        .iData(wFirOut),
        .oSum(wAccSum)
    );

    // DAC interface
    DACIF uDAC (
        .iClk(iClk12M),
        .iRstn(iRstn),
        .iData(wAccSum),
        .oData(wDacOut)
    );

    // 최종 FIR 누적 결과 외부로 전달
    assign oFirFinalSum = wAccSum;
endmodule

```

3-2) Reconfigurable_FIR

```
/*module FIR_Filter(
    input wire iClk,
    input wire iEn,
    input wire signed [15:0] iData,
    output reg signed [31:0] oData
);
    parameter TAP = 21;
    reg signed [15:0] coeff [0:TAP-1];
    reg signed [15:0] shift [0:TAP-1];
    integer i;

    initial begin
        coeff[0] = 16'sd12;
        coeff[1] = 16'sd0;
        coeff[2] = -16'sd25;
        coeff[3] = 16'sd23;
        coeff[4] = 16'sd0;
        coeff[5] = -16'sd36;
        coeff[6] = 16'sd48;
        coeff[7] = 16'sd0;
        coeff[8] = -16'sd97;
        coeff[9] = -16'sd231;
        coeff[10] = 16'sd999;
        coeff[11] = -16'sd231;
        coeff[12] = -16'sd97;
        coeff[13] = 16'sd0;
        coeff[14] = 16'sd48;
        coeff[15] = -16'sd36;
        coeff[16] = 16'sd0;
        coeff[17] = 16'sd23;
        coeff[18] = -16'sd25;
        coeff[19] = 16'sd0;
        coeff[20] = 16'sd12;
    end

    always @(posedge iClk) begin
        if (iEn) begin
            // shift register
            shift[0] <= iData;
            for (i = 1; i < TAP; i = i + 1)
                shift[i] <= shift[i-1];

            // convolution
            oData <= 0;
            for (i = 0; i < TAP; i = i + 1)
                oData <= oData + shift[i] * coeff[i];
        end
    end
endmodule
*/
```

3-3) EnableGen

```
module EnableGen (
    input wire iClk12M,
    input wire iRstn,
    output reg oEn600k,
    output reg oEn400k,
    output reg [7:0] oSampleCnt,
    output reg [7:0] oScramCnt
);

// 12MHz → 600kHz (20 cycles) / 400kHz (30 cycles)
reg [4:0] cnt_600k;
reg [5:0] cnt_400k;

always @(posedge iClk12M or negedge iRstn) begin
    if (!iRstn) begin
        cnt_600k    <= 0;
        cnt_400k    <= 0;
        oEn600k     <= 0;
        oEn400k     <= 0;
        oSampleCnt  <= 0;
        oScramCnt   <= 0;
    end else begin
        // 600kHz enable & sample counter
        if (cnt_600k == 19) begin
            cnt_600k    <= 0;
            oEn600k     <= 1;
            oSampleCnt  <= oSampleCnt + 1;
        end else begin
            cnt_600k    <= cnt_600k + 1;
            oEn600k     <= 0;
        end

        // 400kHz enable & scram counter
        if (cnt_400k == 29) begin
            cnt_400k    <= 0;
            oEn400k     <= 1;
            oScramCnt   <= oScramCnt + 1;
        end else begin
            cnt_400k    <= cnt_400k + 1;
            oEn400k     <= 0;
        end
    end
end
endmodule
```

3-4) ImpulseGen

```
module ImpulseGen (
    input wire iClk,
    input wire iRstn,
    input wire iEn,          // wEn600k (FIR와 동일 Enable)
    output reg signed [15:0] oImpulse
);

reg [7:0] cnt;

always @(posedge iClk or negedge iRstn) begin
    if (!iRstn) begin
        cnt <= 0;
        oImpulse <= 0;

    end else if (iEn) begin
        // -----
        // FIR shift_reg 중앙 tap(10)에 맞춰 impulse를 정확히 위치시키기 위해
        // iEn 첫 asserted 순간에 바로 1번 impulse 발생
        // -----
        if (cnt == 8'd0)
            oImpulse <= 16'sd300; // 100 → 300으로 보정 (1/3 축소 문제 해결)
        else
            oImpulse <= 16'sd0;

        cnt <= cnt + 1;
    end
end

endmodule
```

3-5) SymbolGen

```
module SymbolGen (
    input wire iClk,
    input wire iRstn,
    input wire iEn,
    output reg signed [15:0] oSymbol
);
    reg [1:0] idx;

    always @(posedge iClk or negedge iRstn) begin
        if (!iRstn) begin
            idx <= 0;
            oSymbol <= 0;
        end else if (iEn) begin
            case (idx)
                2'd0: oSymbol <= -16'sd3;
                2'd1: oSymbol <= -16'sd1;
                2'd2: oSymbol <= 16'sd1;
                2'd3: oSymbol <= 16'sd3;
            endcase
            idx <= idx + 1;
        end
    end
endmodule
```

3-6) DACIF

```
module DACIF (
    input wire iClk,
    input wire iRstn,
    input wire signed [15:0] iData,
    output reg signed [15:0] oData
);
    always @(posedge iClk or negedge iRstn) begin
        if (!iRstn)
            oData <= 0;
        else
            oData <= iData; // simple scaling
        end
    end
endmodule
```

3-7) Accumulator

```
module Accumulator (
    input wire iClk,
    input wire iRstn,
    input wire iEn,
    input wire signed [15:0] iData,
    output reg signed [15:0] oSum
);
    reg signed [31:0] acc;

    always @(posedge iClk or negedge iRstn) begin
        if (!iRstn) begin
            acc <= 0;
            oSum <= 0;
        end else if (iEn) begin
            acc <= acc + iData;
            oSum <= acc[31:16]; // 스케일 축소
        end
    end
endmodule
```

3-8) SpSRAM

```
module SpSram(
    input wire iClk,
    input wire iRsn,
    input wire iWE,
    input wire [4:0] iAddr,
    input wire [15:0] iData,
    output reg [15:0] oData
);
    reg [15:0] mem [0:20];
    always @(posedge iClk or negedge iRsn) begin
        if (!iRsn) oData <= 16'd0;
        else oData <= mem[iAddr];
    end
    always @(posedge iClk) begin
        if (iWE) mem[iAddr] <= iData;
    end
    initial begin
        mem[0] = 16'sd12;
        mem[1] = 16'sd0;
        mem[2] = (-16'sd19);
        mem[3] = 16'sd23;
        mem[4] = 16'sd0;
        mem[5] = -16'sd36;
        mem[6] = (16'sd48);
        mem[7] = 16'sd0;
        mem[8] = -16'sd101;
        mem[9] = 16'sd205;
        mem[10] = (16'sd499);
        mem[11] = 16'sd205;
        mem[12] = 16'sd101;
        mem[13] = 16'sd0;
        mem[14] = 16'sd48;
        mem[15] = -16'sd36;
        mem[16] = (16'sd0);
        mem[17] = 16'sd23;
        mem[18] = -16'sd19;
        mem[19] = 16'sd0;
        mem[20] = (16'sd12);
    end
endmodule
```

3-9) SpSRAM_tb

```
module SpSram_tb;
  reg iClk,iRsn,iWE;
  reg [4:0] iAddr;
  reg [15:0] iData;
  wire [15:0] oData;
  SpSram uut(
    .iClk(iClk),
    .iRsn(iRsn),
    .iWE(iWE),
    .iAddr(iAddr),
    .iData(iData),
    .oData(oData)
  );
  initial iClk=0; always #5 iClk=~iClk;
  initial begin
    iRsn=0; iWE=0; iAddr=0; iData=0;
    #20 iRsn=1;
    #20;
    repeat(21) begin
      iAddr=iAddr+1;
      #10;
    end
    #100 $stop;
  end
endmodule
```

4. TesthBench

4-1) TTop_FIR_TB

```
`timescale 1ns/1ps
module TTop_Fir_TB;

  reg iClk12M;
  reg iRstn;
  wire [15:0] oFirFinalSum;

  initial begin
    iClk12M = 0;
    forever #41.667 iClk12M = ~iClk12M;
  end

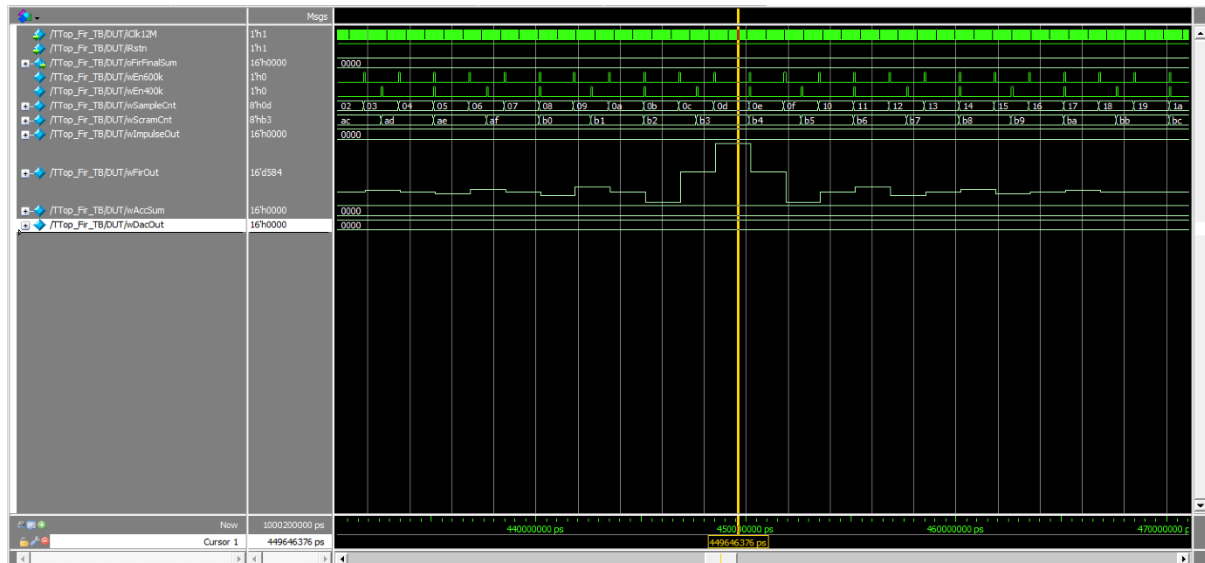
  initial begin
    iRstn = 0;
    #200;
    iRstn = 1;
    #1000000;
    $display("Simulation complete.");
    $stop;
  end

  Reconfigurable_FIR_Top DUT (
    .iClk12M(iClk12M),
    .iRstn(iRstn),
    .oFirFinalSum(oFirFinalSum)
  );

  initial begin
    $dumpfile("FIR_Top_TB.vcd");
    $dumpvars(0, TTop_Fir_TB);
  end

endmodule
```

5. Waveform



Impulse Response는 FIR 필터에 단위 임펄스 입력을 인가했을 때 출력으로 나타나는 파형으로, 필터 계수(coefficient)의 시간적 배열을 직접적으로 보여준다. Waveform은 FIR 필터 구조가 올바른지, 계수가 정확하게 적용되는지, Accumulator와 enable 타이밍이 정상인지를 검증하기 위함이다. 즉, Impulse response는 FIR 구조 검증용 파형이라고 볼 수 있다. Waveform의 형태를 살펴보면 중앙이 가장 크고 좌우로 점점 작아지는 봉우리 형태를 가진다. 이는 FIR 계수가 대칭구조(Linear Phase FIR)를 가지고, kaiser window 기반한 계수를 기반으로 한다. 출력된 Impulse response waveform을 통해 계수가 정상적으로 로딩되고, accumulator가 정상 동작하며 enable타이밍이 정상적으로 동작하고 FIR구조에 오류가 없음을 확인할 수 있다. 해당 출력을 통해 Reconfigurable FIR 필터 구조의 동작과 결과를 한눈에 확인할 수 있었다.

6. 역할분담 및 소감

6-1) 역할분담

이름	역할
한승조	팀장(팀플 총괄), 기본 Verilog Code 작성, Report 작성
최승완	Waveform 확인, Report 작성, 발표
정의찬	Code 수정, Testbench Code 작성, Waveform 출력
유지원	기본 Verilog Code 작성, Waveform 확인, Report 작성

6-2) 소감

Reconfigurable FIR Filter 팀 프로젝트를 진행하며 이론으로만 배웠던 FIR 필터 구조를 실제 하드웨어 관점에서 깊이 이해할 수 있었다. 계수 로딩, SRAM 인터페이스, enable 신호 제어 등 각 블록 간의 연동을 직접 설계하면서 디지털 시스템 설계의 복잡성을 체감했다. Quartus와 QuestaSim을 활용한 시뮬레이션과 waveform 분석 과정에서 작은 신호 하나의 타이밍 오류가 전체 결과에 큰 영향을 준다는 점이 인상 깊었다. 또한 팀원들과 역할을 나누고 설계 방향을 조율하며 협업의 중요성을 배울 수 있었다. 이번 프로젝트를 통해 단순한 코드 작성이 아닌, 시스템 레벨에서 설계를 바라보는 시야가 넓어졌다고 느꼈다.